

PROJECT REPORT

Support Ticket Priority Prediction & Automated Assignment System

Implemented as the SupportIQ AI Ticket Intelligence Dashboard

React 19

Vite

Tailwind 4

Recharts

HIGH

MEDIUM

LOW

Customer Support Ticket Priority Prediction and Automated Assignment System

Delivered as the SupportIQ AI Ticket Intelligence Dashboard

User Story

The Support Ticket Priority Prediction and Automated Assignment System analyses customer support ticket descriptions, automatically determines priority as High, Medium or Low, and triggers automation for appropriate handling. The solution uses a React dashboard, a service layer with an AI classification engine ([analyzeTicket](#)), and an assignment and escalation workflow. High-priority tickets are escalated automatically, tasks are created for urgent handling, and the ticket is assigned to the least-loaded available agent, reducing manual effort and improving response time.

Scope note. This report is based on a static reading of the source code; the application was **not executed**. Where the original template asks for screenshots, dates, team members or test results that do not exist in the source, the section is marked *proposed* or *not yet executed* instead of inventing data.

Contents

1. Introduction	3
2. Ideation Phase	3
3. Requirement Analysis Phase	5
4. Project Design Phase	8
5. Project Planning & Scheduling	10
6. Project Development Phase	11
7. Functional & Performance Testing	16
8. Advantages & Disadvantages	18
9. Conclusion	18
10. Future Scope	18
11. Appendix	19

1. Introduction

1.1 Project Overview

Customer support teams receive a large number of tickets daily, while prioritisation and assignment are often manual. This project provides a dashboard that analyses ticket information, classifies urgency, assigns the appropriate agent, and initiates escalation for critical cases. A support manager can create tickets, see an AI classification (priority, category, SLA risk, confidence), have the ticket auto-assigned, escalate critical incidents, override low-confidence decisions, and watch SLA and workload metrics.

Current maturity: front-end prototype. There is no backend (data lives in `localStorage`), the "AI" is a keyword rule engine, and several dashboard numbers are hard-coded. UI copy references Agentforce and Salesforce Service Cloud, but no Salesforce integration code exists; the wording indicates the intended product direction.

1.2 Purpose

- Automatically classify support tickets as High, Medium or Low priority.
- Show the AI rationale, keyword signals and confidence for every classification.
- Auto-assign each ticket to the least-loaded available agent.
- Automatically escalate High-priority tickets and create the escalation tasks.
- Flag low-confidence (< 70%) tickets for manager review and allow a priority override.
- Monitor SLA risk and agent workload; reduce manual prioritisation and assignment effort.

2. Ideation Phase

2.1 Problem Statement (Customer Problem Statement Template)

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Support Agent	focus on critical tickets first	tickets are not always prioritised automatically	ticket prioritisation can be manual	frustrated
PS-2	Customer	get urgent issues resolved quickly	critical issues may wait behind lower-priority tickets	support teams handle large ticket volumes	concerned
PS-3	Support Manager	assign tickets to the right support level	assignment may require manual decisions	ticket urgency is not consistently analysed	overwhelmed
PS-4	Support Agent	identify urgent issues from descriptions	important keywords can be missed	descriptions contain different urgency signals	uncertain
PS-5	Support Manager	reduce SLA-related risk	older unresolved tickets may remain unnoticed	SLA risk needs additional checking	concerned

2.2 Empathy Map Canvas

User persona: Support Agent / Support Manager.

Empathy Map Canvas - persona: Support Agent / Support Manager

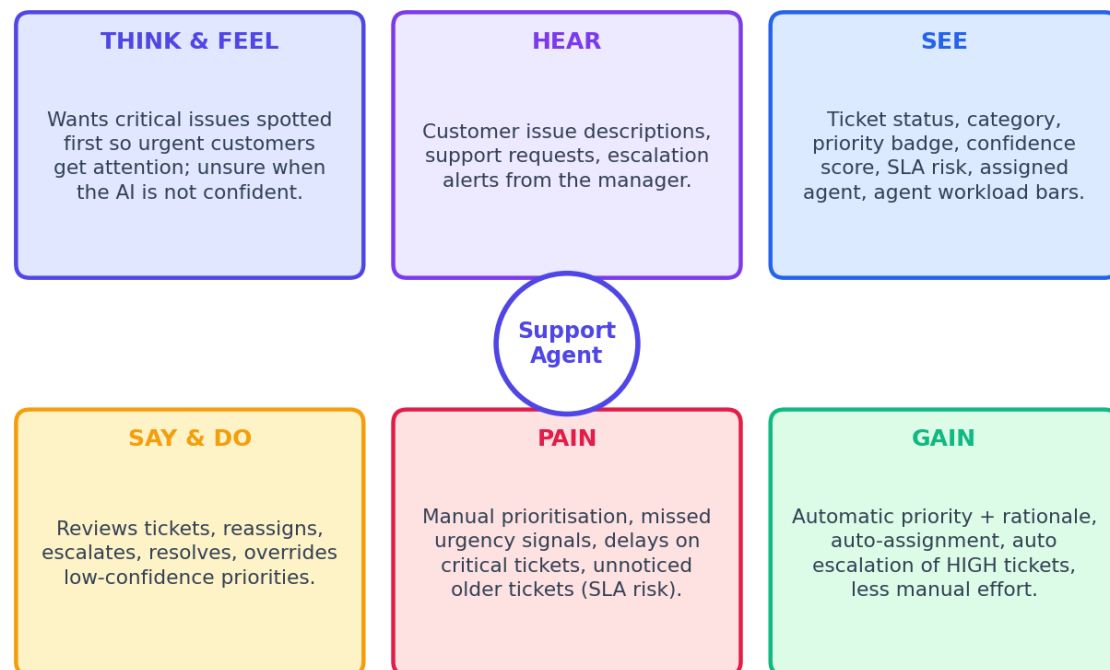


Figure 1 - Empathy map canvas.

2.3 Brainstorming & Idea Prioritisation

Ideas considered

- Manual ticket registers
- Spreadsheet-based ticket tracking
- Standalone support portal
- Salesforce CRM with Agentforce (intended long-term direction)
- React single-page dashboard with a service layer and rule-based AI (prototype)

Final selection

The **React SPA prototype** was built first, because:

- It allows rapid iteration on the triage, escalation and review experience without any backend.
- A single service module (`ticketService`) is the only code that touches data, so mocks can later be swapped for a real API or a CRM connector.
- The AI classifier has a defined output contract that a real model can honour later.
- Automatic priority classification, automatic escalation and reduced manual effort are all demonstrable today.

3. Requirement Analysis Phase

3.1 Customer Journey Map



Stages 1-3 happen inside the Create Ticket page; stages 4-8 continue on the Ticket Details, Escalations and Dashboard pages.

Figure 2 - Journey from ticket creation to resolution.

Stages: Ticket Creation → Description Analysis → Priority & Category → Review → Agent Assignment → Urgent Escalation → Support Handling → Resolution.

3.2 Solution Requirements

Functional requirements

FR No	Requirement	Status in code
FR-1	Support ticket creation and management (create, list, view, resolve)	Implemented
FR-2	Account and Contact association	Implemented (free-text names from mock lists)
FR-3	Ticket description analysis	Implemented (keyword rules)
FR-4	Automatic priority classification (High / Medium / Low)	Implemented
FR-5	High-priority escalation with urgent tasks	Implemented (3 tasks, 2-hour due time)
FR-6	Support agent assignment (auto and manual reassign)	Implemented (least-loaded; ignores skills)
FR-7	SLA breach-risk display	Partial (static snapshot, no timer)
FR-8	Manual review and priority override for low confidence	Implemented (Re-run AI is a no-op)
FR-9	Search, filters and CSV export	Implemented (CSV has a # bug)
FR-10	Conversational (Agentforce) analysis	Not implemented (UI copy only)

Non-functional requirements

NFR	Description	Current state
Usability	Clear AI result panel, rationale, colour-coded badges, toasts	Met
Reliability	Consistent classification from configured rules	Deterministic, but substring matching causes false HIGH
Performance	Quick analysis and rendering	150 ms / 300 ms simulated latency; ~2.5 s analysis animation
Security	Controlled access to customer and ticket data	Not met: no authentication or roles
Maintainability	Rules and service layer can be updated in one place	Good: single service module; no tests
Scalability	Supports growing ticket volume	Limited by <code>localStorage</code> ; needs a backend

3.3 Data Flow Diagram

Data flow diagram

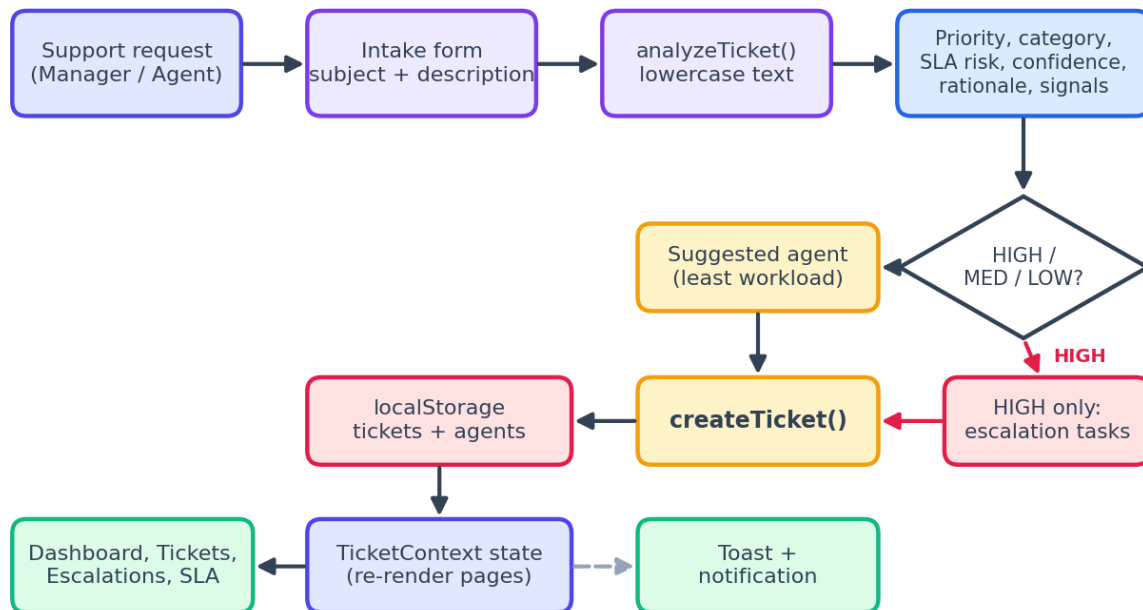


Figure 3 - Data flow from support request to updated dashboards.

Flow: Support request → Intake form → `analyzeTicket` → priority, category, SLA risk, confidence → High / Medium / Low decision → suggested agent → `createTicket` → (High only) escalation tasks → `localStorage` → context state → pages, toast and notification.

3.4 Technology Stack

Layer	Technology
Platform	Browser single-page application (Vite + React 19)
Routing	react-router-dom 7
Database	Browser <code>localStorage</code> (<code>supportiq_tickets</code> , <code>supportiq_agents</code>), seeded from <code>mockData.js</code>
Logic	<code>ticketService</code> (async service layer)
AI	Keyword rule engine <code>analyzeTicket()</code>
State	React Context (<code>TicketContext</code> , <code>ToastContext</code>)
Styling & charts	Tailwind CSS 4, Recharts 3, lucide-react
Tooling	Vite 8, oxlint
Output	Dashboard pages, toasts, notification panel, CSV export

4. Project Design Phase

4.1 Problem-Solution Fit

Problem	Solution	Benefit
Manual ticket prioritisation	<code>analyzeTicket</code> reads the description and sets priority	Faster identification of urgent cases
Urgent issues may be missed	High-signal keyword rule (outage, urgent, 502 ...)	Critical cases receive attention
Manual task creation	Escalation attaches three tasks automatically	Reduced manual effort
Manual support assignment	Least-loaded available agent is suggested and assigned	More consistent handling
SLA risk can be overlooked	SLA risk level, remaining time and SLA Monitor page	Better risk awareness
AI may be wrong	Confidence score and < 70% manual-review queue with override	Human stays in control

4.2 Proposed Solution

Parameter	Description
Problem	Manual ticket prioritisation and assignment cause delays in handling critical customer support issues.
Solution	A support-operations console that classifies, routes, escalates and monitors tickets.
Innovation	AI-style classification with rationale, keyword evidence and confidence; automatic escalation; a manual-review gate for low-confidence results.
Impact	Faster identification of critical tickets, reduced manual effort, improved response time and better team productivity.
Business model	Customer support operations involving customers, support agents, support managers and AI-assisted ticket handling.
Scalability	Service-layer boundary allows a real REST backend, a real classifier/LLM, richer assignment rules and live SLA timers.

4.3 Solution Architecture

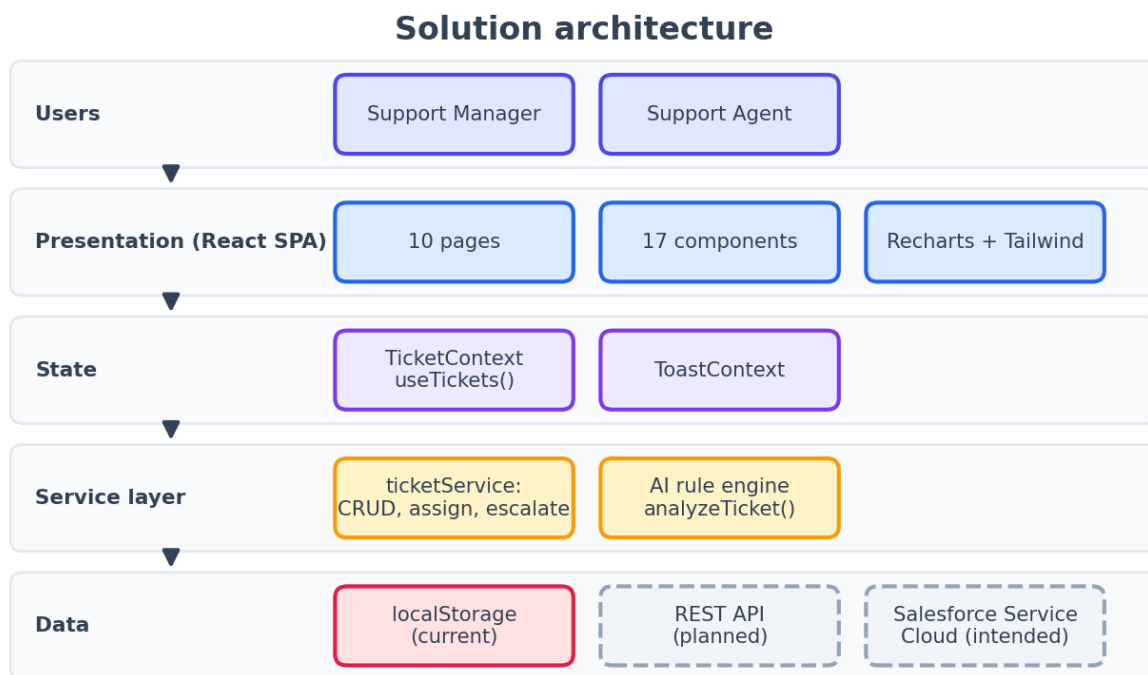
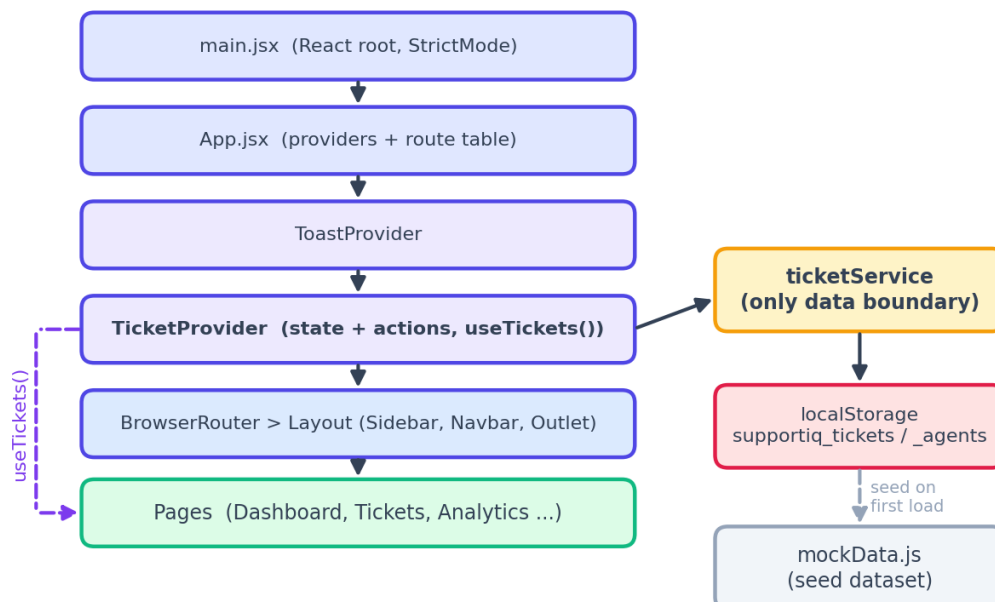


Figure 4 - Layered architecture. Dashed boxes are planned or intended, not implemented.

Components: Users → React pages and components → TicketContext → ticketService (rule engine, assignment, escalation) → data store. Provider order matters: `ToastProvider` wraps `TicketProvider`, which wraps the router and layout.

Provider and data flow



All service methods are async with an artificial 150 ms delay (300 ms for analysis).

Figure 5 - Provider hierarchy and the single data boundary.

5. Project Planning & Scheduling

5.1 Project Planning

- Agile methodology
- Epics → Stories → Story points
- Sprint-based execution
- Velocity calculation used for estimation

Proposed plan. The repository contains no sprint records, so the backlog below is *proposed* from the production roadmap and the known-issues list. Team members, dates and actual completion figures are left as TBD for the team to fill in.

Product backlog, sprint schedule and estimation

Sprint	Epic	USN	User story / task	SP	Priority	Team
1	Truthful UI	USN-1	As a manager, I want every KPI and chart derived from live tickets so the numbers can be trusted.	8	High	TBD
1	Truthful UI	USN-2	As an admin, I want settings (threshold, max open, algorithm) applied to analysis and assignment and persisted.	5	High	TBD
1	Truthful UI	USN-3	As a manager, I want resolving a ticket to lower the agent's workload and remove it from active escalations.	5	High	TBD
1	Truthful UI	USN-4	As an agent, I want Re-run AI to actually re-analyse a ticket.	2	Medium	TBD
2	Backend integration	USN-5	As a developer, I want <code>ticketService</code> to call a REST API so data is shared across users.	8	High	TBD
2	Backend integration	USN-6	As a developer, I want ticket IDs and workload computed server-side to avoid collisions.	5	High	TBD
2	Classification quality	USN-7	As an agent, I want whole-word keyword matching so "not critical" or "15024" is not marked HIGH.	5	High	TBD
2	Export	USN-8	As a manager, I want CSV export via Blob so subjects containing # export correctly.	2	Medium	TBD
3	AI model	USN-9	As a manager, I want a real classifier/LLM returning the same output contract with genuine confidence.	8	High	TBD
3	Assignment	USN-10	As a manager, I want assignment to consider skills, capacity and exclude the manager.	5	High	TBD
3	SLA	USN-11	As a manager, I want live SLA countdown timers so risk escalates automatically.	5	Medium	TBD
3	Workflow	USN-12	As an agent, I want an action that moves a ticket to IN_PROGRESS.	2	Medium	TBD
4	Security	USN-13	As an admin, I want authentication and roles (agent vs manager).	8	High	TBD
4	Notifications	USN-14	As a user, I want persistent, real-time notifications that respect my toggles.	5	Medium	TBD
4	Quality	USN-15	As a developer, I want automated tests and CI to protect the rules.	5	High	TBD
4	Quality	USN-16	As a developer, I want a TypeScript migration started for the service contract.	2	Low	TBD

Project tracker, velocity & burndown

Sprint	Total story points	Duration	Start date	End date (planned)	Points completed	Release date (actual)
Sprint-1	20	TBD	TBD	TBD	TBD	TBD
Sprint-2	20	TBD	TBD	TBD	TBD	TBD
Sprint-3	20	TBD	TBD	TBD	TBD	TBD
Sprint-4	20	TBD	TBD	TBD	TBD	TBD

Planned velocity = 80 points / 4 sprints = **20 points per sprint**. Actual velocity is recorded once sprints are executed.

Path to production

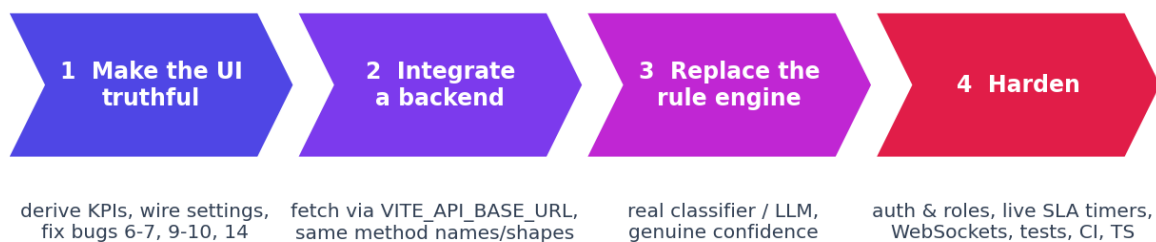


Figure 6 - The four sprints follow the four production-readiness phases.

6. Project Development Phase

What you'll learn

- Data modelling (tickets, agents, settings, notifications)
- Service-layer design and async data access
- Rule-based AI classification and its limitations
- React Context state management
- Component-driven dashboards with Tailwind and Recharts
- Routing, search and filtering

Milestone 1: Development Environment

Activity 1: Install prerequisites

Install a current Node.js LTS and npm, then clone or extract the project.

Activity 2: Run the application

```

npm install      # install dependencies
npm run dev      # Vite dev server with HMR
npm run build    # production build to dist/
npm run preview  # serve the production build
npm run lint     # oxlint
  
```

The app seeds 12 tickets and 6 agents into `localStorage` on first load. **Settings** → **Reset** restores the seed data.

Milestone 2: Data Management - Ticket and Agent Records

Ticket record (equivalent of the Support Ticket Intelligence object)

Field label	Key	Data type	Description
Ticket Number	id	string	STI-NNNNN, generated as <code>tickets.length + 1</code>
Customer	account	string	Related account name (mock list)
Contact	contact	string	Customer contact (mock list)
Issue Type	issueCategory	string	System Outage, Performance, Account Access, Billing, General Inquiry
Description	description	string	Issue details
Priority Level	priority	enum	LOW, MEDIUM, HIGH
Status	status	enum	OPEN, IN_PROGRESS, ESCALATED, RESOLVED
Created Date	createdDate	ISO string	Ticket date
Assigned To	assignedAgent	string	Agent name
SLA Breach Risk	slaRiskLevel	enum	LOW, MEDIUM, HIGH, BREACHED
Resolution Time (hrs)	resolutionTimeHours	number	Time taken
AI Confidence	aiConfidenceScore	number 0-100	Classifier confidence
Manual Review	requiresManualReview	boolean	True for low-confidence tickets

Agents carry `id`, `name`, `availability`, `openTickets`, `highPriority`, `workload` (0-100) and `skills[]`. Full field tables are in Appendix A.

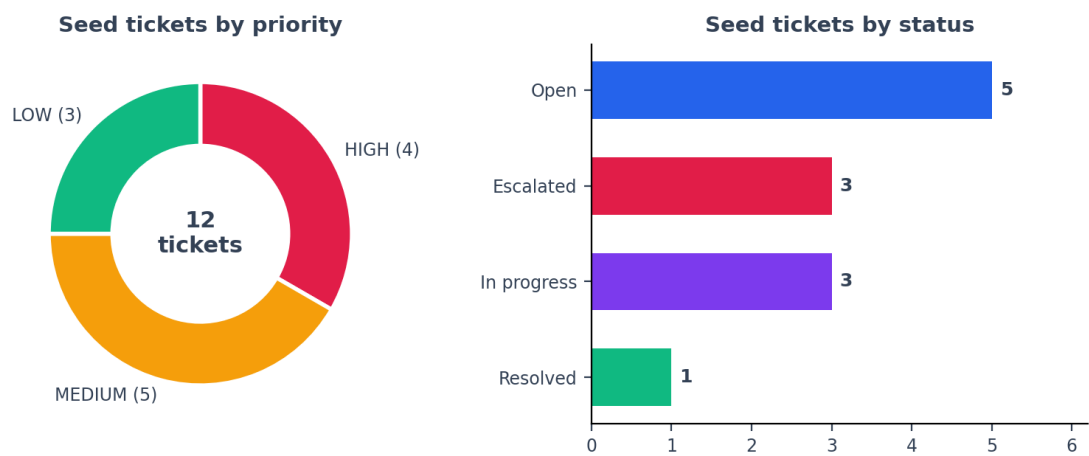


Figure 7 - Seed dataset: 12 tickets by priority and status.

Milestone 3: Classification and Assignment Logic

`ticketService.analyzeTicket(subject, description)` plays the role of the decision logic. It takes the ticket text as input and returns the output contract below.

Variable	Direction	Type	Description
subject, description	Input	string	Ticket text (lowercased and combined)
predictedPriority	Output	enum	HIGH, MEDIUM or LOW
issueCategory	Output	string	Detected category
slaRisk	Output	enum	LOW, MEDIUM or HIGH
aiConfidenceScore	Output	number	62, 86, 93 or 96 (fixed per branch)

Variable	Direction	Type	Description
aiRationale, keywordSignals	Output	string, string[]	Explanation and up to 5 detected terms
requiresManualReview	Output	boolean	True for vague tickets
suggestedAgent	Output	string	Available agent with the lowest workload

Step-by-step logic

1. **Normalise:** lowercase `subject + description`.
2. **Vague check:** if `description.trim().length < 35` and no keyword is found, return MEDIUM / LOW risk / 62% / General Inquiry with manual review.
3. **High path:** if any high signal is present (outage, system down, data loss, security breach, urgent, critical, emergency, asap, 502-504, crash, compromised), return HIGH / HIGH / 96%; category becomes Billing for payment/checkout/stripe, or a security-flavoured System Outage.
4. **Low path:** if any low signal is present (question, how to, information, documentation, upgrade, pricing, seats, invoice), return LOW / LOW / 93%.
5. **Default:** otherwise MEDIUM / MEDIUM / 86%; category is Account Access for login/sso/auth, else Performance.
6. **Suggest agent:** among `Available` agents choose the lowest workload; fallback `Priya Sharma`.
7. **Create ticket:** apply SLA target (HIGH 2 h, MEDIUM 8 h, LOW 24 h); HIGH tickets start as `ESCALATED`.
8. **Escalate (HIGH):** attach `escalationDetails` with three tasks (Review and respond, Manager outage escalation, Incident Bridge setup), notify agent and manager, due "Today - 2 hours".
9. **Update workload:** `workload = round(openTickets / 25 * 100)`, capped at 100.
10. **Feedback:** success or warning toast and, for HIGH, a notification.

AI rule engine: first match wins

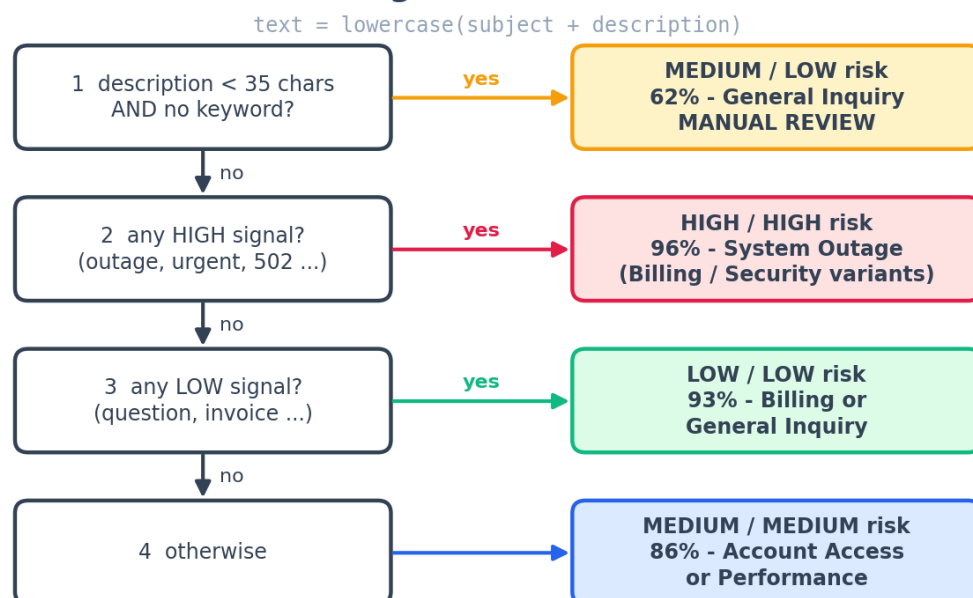


Figure 8 - Rule order: the first match wins.

Final flow structure

Create-ticket workflow

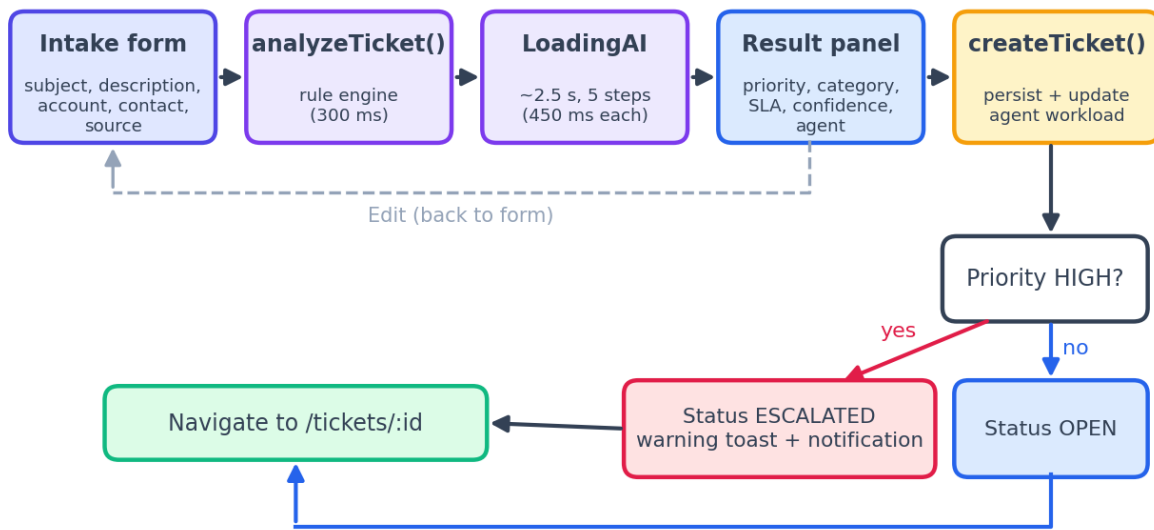


Figure 9 - Create-ticket workflow.

Start → analyzeTicket → Priority decision → Suggest agent → createTicket → (High)
 Escalate → Update workload → Toast / notification → End

Ticket status transitions implemented in code

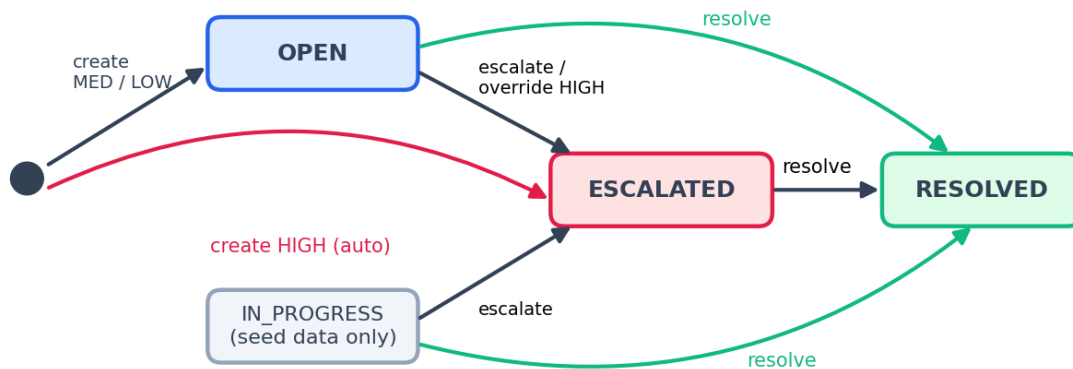


Figure 10 - Ticket status transitions.

Milestone 4: State Management

`TicketContext` owns `tickets`, `agents`, `settings`, `notifications`, `loading` and `searchQuery`, and exposes actions through `useTickets()`: `createTicket`, `assignTicket`, `escalateTicket`, `overridePriority`, `resolveTicket`, `refreshData`, `resetAllData`. After each action the affected ticket is replaced with the object returned by the service.

Action	Effect
<code>createTicket(data)</code>	Persists ticket, updates workload, toasts, notifies on HIGH
<code>assignTicket(id, agent)</code>	Reassigns and recomputes both agents' workloads
<code>escalateTicket(id, reason)</code>	Status ESCALATED, priority HIGH, tasks attached

Action	Effect
<code>overridePriority(id, priority, reason)</code>	Records override; HIGH also escalates
<code>resolveTicket(id)</code>	Status RESOLVED with resolved date

Milestone 5: User Interface

Ten routes sit inside a shared layout (sidebar, navbar, outlet). The wireframes below are **schematic illustrations**, not screenshots.

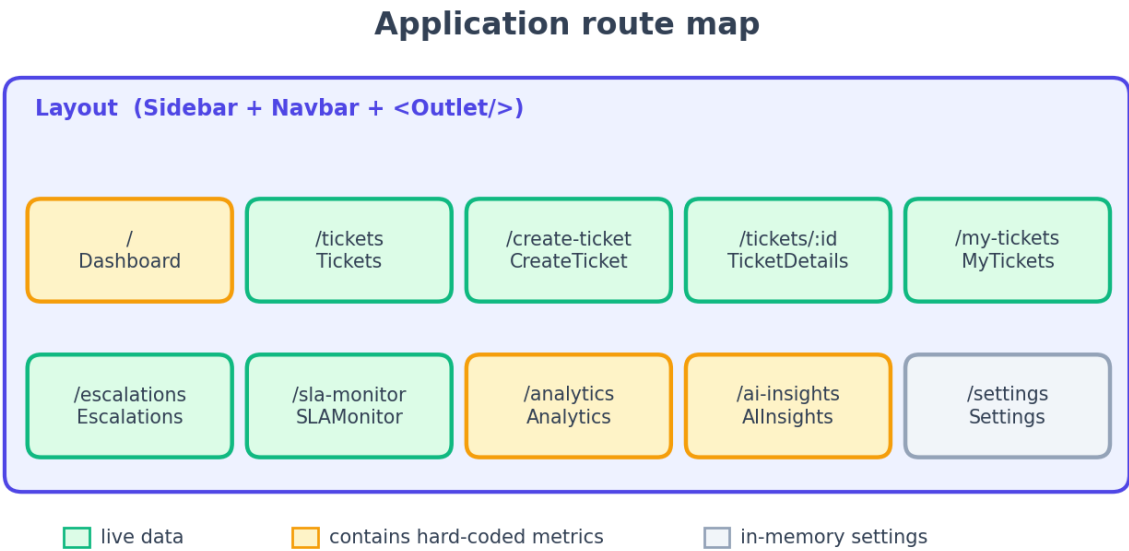


Figure 11 - Route map.

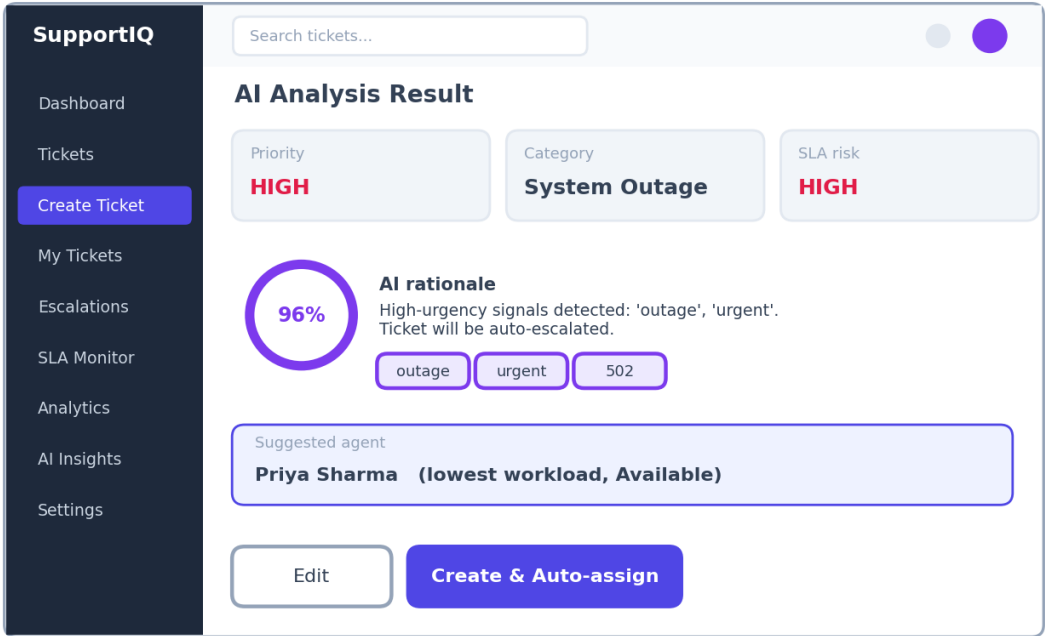


Figure 12 - Schematic wireframe: Create Ticket result panel (not a screenshot).

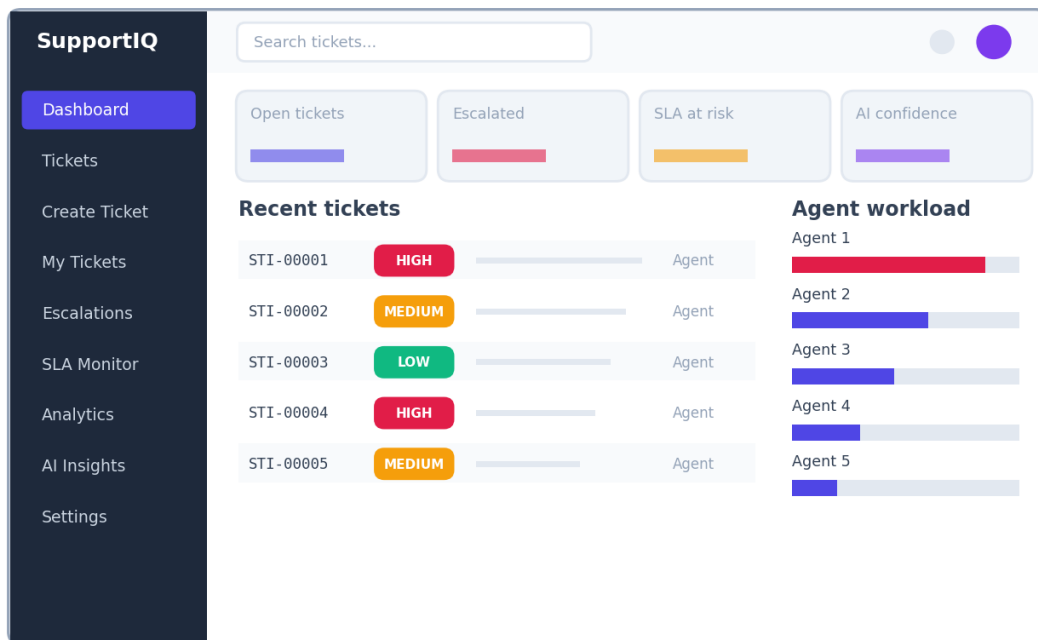


Figure 13 - Schematic wireframe: Dashboard (not a screenshot).

Final outcome

- Takes subject, description, account and contact as input
- Analyses the description using keyword rules
- Sets priority, category, SLA risk and confidence automatically
- Suggests and assigns the least-loaded available agent
- Escalates High tickets with tasks and notifications
- Flags low-confidence tickets for manager review and override

7. Functional & Performance Testing

Not yet executed. The application was not run for this report, so no screenshots or measured results are claimed. The table lists test cases with the **expected result derived from reading the code**. Run each case in the app and record the actual result before submission.

7.1 Functional test cases

TC	Input / action	Expected result (from code)	Actual
1	"Production outage, system down, urgent" (long description)	HIGH, 96%, System Outage, status ESCALATED, warning toast + notification	TBD
2	"Urgent: checkout payment failing for all customers"	HIGH, 96%, category Billing	TBD
3	"Security breach, token compromised"	HIGH, 96%, System Outage (security wording)	TBD
4	Description "Not sure what is wrong here." (< 35 chars, no keyword)	MEDIUM, 62%, General Inquiry, manual review flagged	TBD
5	"How do I upgrade my plan? Need pricing for more seats"	LOW, 93%, Billing	TBD
6	"Login via SSO keeps failing after the auth token expired"	MEDIUM, 86%, Account Access	TBD
7	"Dashboard is slow with lag and frequent timeout"	MEDIUM, 86%, Performance	TBD
8	Override a 62% ticket to HIGH with a reason	Priority HIGH, <code>priorityOverridden</code> , review flag cleared, ticket escalated	TBD
9	Reassign a ticket to another agent	Old agent open count decrements, new increments, both workloads recomputed	TBD
10	Resolve a ticket	Status RESOLVED, resolved date set, success toast	TBD
11	Search "Priya" in the top bar	Navigates to /tickets and filters by assigned agent	TBD
12	Settings → Reset	Seed data of 12 tickets and 6 agents restored	TBD

7.2 Edge cases that expose known defects

TC	Input / action	Expected (current) behaviour	Defect
13	"This is not critical, just a cosmetic question"	Classified HIGH (substring match on <i>critical</i>)	#8
14	"Order 15024 is delayed"	Classified HIGH (number contains 502)	#8
15	"Urgent question about pricing"	HIGH (high beats low)	limitation
16	Resolve a ticket, then view agent workload	Agent's open count does not drop	#6
17	Resolved HIGH ticket on Escalations page	Still listed as an active escalation	#7
18	Export CSV with # in a subject	File may be truncated at #	#9
19	Click Re-run AI in the review modal	Nothing happens (empty handler)	#14
20	Change a setting then reload the page	Setting is lost and was never applied	#2

7.3 Performance observations

Item	Value (defined in code)
Service latency	150 ms per call; 300 ms for analysis (simulated)
AI loading animation	5 steps x 450 ms = ~2.5 s, independent of the analysis call
Toast auto-dismiss	4 s
Data scale	Limited by <code>localStorage</code> quota; all filtering is client-side

Real performance should be measured with the browser's performance tools and a larger seeded dataset.

8. Advantages & Disadvantages

Advantages

- **Automatic ticket prioritisation** - classifies tickets as High, Medium or Low from the description.
- **Faster identification of critical issues** - agents can focus on urgent tickets first.
- **Transparent AI** - every result shows rationale, keyword signals and a confidence score.
- **Human-in-the-loop** - tickets below 70% confidence go to a manual-review queue with override.
- **Automatic escalation** - High tickets are escalated with tasks and notifications.
- **Consistent assignment** - least-loaded available agent, with manual reassign.
- **Operational visibility** - SLA risk, agent workload, filters, search and CSV export.
- **Clean architecture** - one service module is the only data boundary, easing the move to a real backend.
- **Reduced manual effort** - less repetitive triage work for support teams.

Disadvantages

- **Prototype only** - no backend; data is per-browser `localStorage`.
- **Keyword-based priority logic** - substring matching causes false HIGH results and ignores context.
- **Fixed confidence values** - 62, 86, 93 and 96 are constants, not computed.
- **Hard-coded metrics** - Dashboard, Analytics and parts of AI Insights do not reflect live tickets.
- **Settings not applied** - thresholds, limits and notification toggles have no effect and are not persisted.
- **No authentication or roles** - the current user is hard-coded.
- **Workload and escalation bugs** - resolving does not reduce workload; resolved HIGH tickets stay listed.
- **Static SLA** - remaining time never counts down.
- **Assignment ignores skills and capacity** and can pick the Support Manager.
- **No automated tests**, limited linting and no TypeScript.
- **No Salesforce or Agentforce integration** despite the UI wording.

9. Conclusion

SupportIQ demonstrates how rule-based AI, automated routing and a manual-review gate can reduce manual support-ticket handling. The prototype analyses a ticket description, classifies priority, category and SLA risk, assigns the least-loaded available agent, escalates High-priority cases with urgent tasks, and gives managers visibility into workload and SLA risk. This supports faster ticket handling, reduced manual effort and better team productivity.

It is, however, a front-end prototype. Before production use it needs truthful metrics, applied settings, a backend, a real classifier, and authentication, as outlined in the future scope.

10. Future Scope

- Derive every KPI and chart from live data; wire and persist settings.
- Integrate a REST backend behind the existing `ticketService` method names and return shapes.
- Replace the rule engine with a real classifier or LLM that honours the same output contract, with genuine confidence.
- Use whole-word matching and add skills- and capacity-aware assignment rules.
- Add live SLA timers, escalation rules and real-time notifications.

- Add authentication and roles (agent vs manager) and an audit log.
- Introduce analytics for priority trends and resolution performance.
- Connect to Salesforce Service Cloud / Agentforce: Account, Contact, Task and Case Owner objects map naturally to the current fields.
- Add automated tests, CI and TypeScript.

Proposed endpoint	Maps to
GET /tickets?search&priority&status&slaRisk&category&agent&manualReviewOnly	getTickets
POST /tickets/analyze	analyzeTicket
POST /tickets	createTicket
POST /tickets/{id}/assign	assignTicket
POST /tickets/{id}/escalate	escalateTicket
POST /tickets/{id}/priority-override	overridePriority
GET /agents/workload	getAgentWorkload

Proposed only - no such backend exists in this repository.

11. Appendix

A. Ticket, Agent, Settings and Notification records

Record	Fields
Ticket	id, subject, description, priority, status, issueCategory, slaRiskLevel, slaTargetHours, slaRemainingMinutes, aiConfidenceScore, aiRationale, keywordSignals, requiresManualReview, priorityOverridden, overriddenBy, overrideReason, aiProcessed, assignedAgent, account, contact, ticketSource, resolutionTimeHours, createdAt, escalatedDate, resolvedDate, escalationDetails
Agent	id, name, avatar, email, role, availability, openTickets, highPriority, workload, skills[], statusText
Settings	confidenceThreshold (70), maxOpenTickets (20), assignmentAlgorithm (least-loaded), four notification toggles, userAvailable, escalationTierAgent
Notification	id, title, message, time, unread, type (escalation, breach, review, assignment)

B. Key records and storage

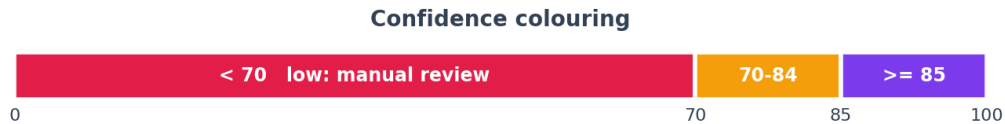
Tickets and agents: `localStorage` keys `supportiq_tickets` and `supportiq_agents`. `supportiq_settings` is defined but unused. Seed roster: Priya Sharma, Rahul Kumar, Anita Joseph, Arun Kumar, Meena Devi, Kathirvel (Support Manager).

C. Service: ticketService

Method	Purpose
getTickets, getTicketById	List and fetch tickets
analyzeTicket	Rule-engine classification
createTicket, updateTicket	Persist and merge updates
assignTicket, escalateTicket, overridePriority	Workflow actions
getAgentWorkload	Agent list with workload
resetToDefaultData	Reseed storage

D. AI Classification Engine: signal vocabularies

- **High:** outage, system failure, system down, data loss, security breach, urgent, critical, emergency, asap, 502, 503, 504, crash, compromised
- **Medium:** slow, lag, timeout, performance, login, unable to login, access, auth, expired, error, bug, failing
- **Low:** question, how do i, how to, information, documentation, feature inquiry, upgrade, pricing, seats, invoice



Confidence colouring used by the UI.

E. State API and components

`useTickets()` exposes tickets, agents, settings, notifications and the actions listed in Milestone 4.

`useToast()` provides `addToast(message, type, duration)` and `removeToast(id)`. Shared components: Layout, Sidebar, Navbar, NotificationPanel, StatCard, TicketTable, TicketCard, PriorityBadge, StatusBadge, SLABadge, ConfidenceScore, AIAnalysisCard, LoadingAI, ManualReviewModal, EscalationPanel, AgentWorkload, EmptyState.

Thank You